

Developer Guide

Rotana Spa Management System — how to build, extend and maintain.

1. Technology stack

Layer	Technology
Framework	Next.js 16.2 (App Router, Turbopack dev server)
UI	React 19, TypeScript, Bootstrap 5 + custom CSS (<code>app/globals.css</code>)
i18n	next-intl (English <code>en</code> , Amharic <code>am</code>)
Database	PostgreSQL 15+ (<code>pg</code> pool), managed Neon in this environment
Auth	JWT (bearer token) + bcrypt password hashing
Printing	80 mm / 58 mm thermal slip rendered in the browser (print)
Deploy	Node server (<code>next start</code>) or PM2 (<code>ecosystem.config.js</code>); Vercel-compatible

Note: This project uses a customized Next.js 16 build. Before writing app code, read the relevant guides in `node_modules/next/dist/docs/` and heed deprecation notices (for example, the `middleware.ts` → `proxy` file convention).

2. Project structure

```
rotana-spa-management/
├─ app/
│  ├─ [locale]/                # next-intl locale segment (en, am)
│  │  ├─ dashboard/           # authenticated UI pages
│  │  │  ├─ membership/      # classic membership screens
│  │  │  ├─ spa/...          # operations, gym, spa, customers, inventory, staff, facilities, reports
│  │  │  ├─ users/ roles/ settings/ audit/ notifications/
│  │  │  ├─ admin/ companies/ demo-licenses/ audit-logs/  # super admin
│  │  │  └─ page.tsx          # dashboard
│  │  └─ login/ page.tsx      # login entry
│  └─ api/                    # route handlers (see API_DOCUMENTATION.md)
│     ├─ auth & login/
│     ├─ membership/...      # plans, members, subscriptions, rate-cards, facilities, gates, attendance, ...
│     └─ spa/
│        └─ [section]/[module]/ # dynamic operational workspace CRUD
```

```

| | | └─ visits/ + [id]/services + [id]/service-order
| | | └─ service-orders/
| | | └─ reports/[report]/
| | └─ ... # users, roles, companies, settings, notifications, audit-logs, ...
| └─ layout.tsx
| └─ globals.css
└─ components/ # DataTable, StatCard, StatusBadge, FormField, ConfirmDialog, ...
└─ lib/
| └─ db.ts # pg pool + env config
| └─ auth.ts / api-utils.ts # auth helpers, withAuth/ok/err/created/badRequest
| └─ permissions.ts # requirePermission, can()
| └─ permission-defs.ts # resource groups
| └─ spa-modules.ts # dynamic module & report definitions
| └─ spa-navigation.ts # sidebar groups/links
| └─ spa-service-orders.ts # order snapshot types
| └─ id-generator.ts # sequential/patterned IDs
| └─ audit.ts # audit log writer
| └─ cache.ts / storage.ts / gem-ui.tsx
└─ messages/ # en.json, am.json
└─ i18n/request.ts # locale resolution
└─ scripts/ # seed, CRUD/API tests, dev helpers
└─ docs/ # this documentation set
└─ database/ # DB doc pointers

```

3. Setup (local development)

```

# 1. install
npm install

# 2. environment (.env) – see DEPLOYMENT.md
DATABASE_URL=postgresql://user:pass@host/db?sslmode=require
JWT_SECRET=<long-random-secret>

# 3. apply migrations in order
psql "$DATABASE_URL" -f db-migration.sql
# ... v2 .. v34

# 4. seed accounts (if fresh)
node scripts/seed-super-admin.js

# 5. run dev server
npm run dev # http://localhost:3000

```

Optional demo data (idempotent):

```
node scripts/_seed-dagi.cjs
```

4. Key conventions

API route handler shape

Every API route follows this pattern:

```
import { withAuth, ok, err, badRequest } from "@lib/api-utils";
import { requirePermission } from "@lib/permissions";

export async function POST(req: Request) {
  return withAuth(req, async (user) => {
    const perm = await requirePermission(user, "create", "spa_visits");
    if (!perm.allowed) return NextResponse.json({ error: "Permission denied" }, { status: 403 });
    try {
      // ... parameterised SQL ...
      return created(row);
    } catch (e: any) {
      return err(e.message);
    }
  });
}
```

Tenancy

- All tenant tables carry `company_id`; every query must scope `WHERE company_id = $n`.
- Super admins have `company_id = NULL`; they must pass an explicit `company_id` to create tenant records. Helper pattern:

```
const companyId = user.role === "super_admin"
  ? Number(rawBody.company_id) || null
  : user.company_id;
```

- Ownership guard for updates/deletes:

```
WHERE id = $1 AND ($2 = true OR company_id = $3)  -- $2 = (role === 'super_admin')
```

Dynamic workspaces (`/api/spa/[section]/[module]`)

- Module definitions live in `lib/spa-modules.ts` (`SPA_MODULES`).
- Fields are validated against the definition; values are stored in the `details` jsonb column of `spa_management_records`.

- Add a workspace by adding a `moduleDefinition({...})` entry + a sidebar link in `lib/spa-navigation.ts` + permission rows in the matching migration. No new route or table is needed.
- Derived-status logic (BMI, commission, stock level, class full, towel state) lives in `applyDerivedValues()` inside the dynamic route.

Reports

- Report definitions live in `lib/spa-modules.ts` (`SPA_REPORTS`).
- The single handler `app/api/spa/reports/[report]/route.ts` renders summary + columns + rows for `membership` , `attendance` , `service-orders` , `therapist` , `trainer` , `inventory` .

IDs

- `generateSequentialId(table, column, prefix)` computes the next `PREFIX-###` by scanning the max existing value (used for `MEM-###` , `CMP-###`).
- Visit numbers use a per-company counter table `spa_visit_counters` (`SPA-000001`).
- Service order numbers use a per-company sequence (`SO-SPA-000001`).

Audit

- Call `logAudit({ company_id, user_id, action, table_name, record_id, old_values?, new_values? })` after mutating writes. It must not fail the main transaction (best-effort).

5. Adding a feature — recipe

1. **DB:** create `db-migration-v<N+1>.sql` (idempotent `CREATE TABLE IF NOT EXISTS`), apply it.
2. **API:** add route under `app/api/...` following the `withAuth` + permission + parameterised SQL pattern.
3. **UI:** add a page under `app/[locale]/dashboard/...` reusing the shared components.
4. **Nav:** add a `SidebarLink` in `lib/spa-navigation.ts` with its permission resource.
5. **Permissions:** add `role_permissions` rows in the migration for the roles that need access.
6. **i18n:** add keys to `messages/en.json` and `messages/am.json` .
7. **Test:** run `scripts/_crud-test.cjs` (66 checks) and, for visit/service-order changes, `scripts/_spa-workflow-test.cjs` (12 checks).

6. Testing

Command	Purpose
<code>npx tsc --noEmit</code>	Type check (run with dev server stopped / <code>.next</code> clean to avoid generated-file noise).
<code>npm run lint</code>	ESLint.
<code>node scripts/_crud-test.cjs</code>	66-point CRUD/API suite against the running server (expects <code>admin@rotanaspa.com</code>).

Command	Purpose
<code>node scripts/_spa-workflow-test.cjs</code>	12-point visit → service-order handoff flow.
<code>node scripts/_seed-dagi.cjs</code>	Idempotent demo dataset (also a smoke test of many endpoints).

7. TypeScript notes

- Route params are awaited: `const { id } = await params;` (Next 16 App Router).
- `tsserver` may report errors inside `.next/dev/types/*` generated files while the Turbopack dev server runs; these are regenerated artifacts, not source errors. Verify with `npx tsc --noEmit` against a clean `.next` when in doubt.
- Avoid dynamic SQL string interpolation of column names from user input; whitelist fields explicitly (see the plans/gates/facilities/rate-cards PUT handlers for the allowed-field pattern).